

# Class06: R Functions

Joli DeLucia (A17320476)

## Table of contents

|                               |   |
|-------------------------------|---|
| Background . . . . .          | 1 |
| A first function . . . . .    | 1 |
| A second function . . . . .   | 3 |
| A new cool function . . . . . | 4 |

## Background

Functions are at the heart of using R. Everything we do involves calling and using functions (from data, input, analysis to results output).

All functions in R have at least 3 things:

1. A **name** the thing we use to call the function.
2. One or more input **arguments** that are comma separated
3. The **body**, lines of code between curly brackets { } that does the work of the function

## A first function

Let's write a silly function that adds some numbers:

```
add <- function(x) {  
  x + 1  
}
```

Let's try it out

```
add(100)
```

```
[1] 101
```

Will this work

```
add( c(100,200, 300) )
```

```
[1] 101 201 301
```

Modify to be more useful and add more than just 1

```
add <- function(x, y = 1) {  
  x + y  
}
```

Let's try it

```
add(100, 10)
```

```
[1] 110
```

Will this work?

```
add(100)
```

```
[1] 101
```

```
log(10, base = 10)
```

```
[1] 1
```

**N.B** Input arguments can be either **required** or **optional**. The later have a fall-back default that is specified in the function code with the equals sign =.

```
#add(100, 200, 300)
```

## A second function

All functions in R look like this

```
name <- function(arg) {  
  body  
}
```

The `sample()` function in R ...

```
sample(1:10, 4)
```

```
[1] 7 1 8 9
```

Q. Return 12 numbers picked randomly from the input 1:10

```
sample(1:10, 12, replace = TRUE)
```

```
[1] 2 10 4 1 9 1 9 3 10 1 2 7
```

Q. Write the code that generates a random 12 nucleotide long DNA sequence?

```
sample( c("A", "T", "G", "C"), 12, replace = TRUE)
```

```
[1] "G" "T" "G" "G" "G" "C" "A" "G" "A" "G" "A" "T"
```

Q. Write a first version function called `generate_dna()` that generates a user specified length `n` random DNA sequence?

```
generate_dna <- function(n = 6) {  
  sample( c("A", "T", "G", "C"), size = n, replace = TRUE)  
}
```

```
generate_dna(15)
```

```
[1] "A" "A" "G" "G" "A" "C" "A" "C" "A" "G" "A" "C" "A" "G" "G"
```

Q. Modify your function to return a FASTA like sequence so rather than [1] “G” “A” “C” “G” we want “GACG”

```
generate_dna <- function(n = 6) {
  paste(sample( c("A", "T", "G", "C"), size = n, replace = TRUE), collapse = "")
}
```

```
generate_dna(15)
```

```
[1] "TCACCGTAGGCACTG"
```

Q. Give the user the option to return the FASTA format output or the standard multi-element vector format?

```
generate_dna <- function(n = 6, fasta = TRUE) {
  ans <- sample( c("A", "T", "G", "C"), size = n, replace = TRUE)

  if(fasta) {
    ans <- paste(ans, collapse = "")
  }

  return(ans)
}
```

```
generate_dna(15, fasta = F)
```

```
[1] "G" "C" "T" "A" "C" "G" "G" "G" "G" "T" "A" "A" "G" "C" "G"
```

```
generate_dna(15, fasta = T)
```

```
[1] "CCTTCATGACAGATC"
```

```
generate_dna(15)
```

```
[1] "GTTCTTCGTTTTAGG"
```

## A new cool function

Q. Write a function called `generate_protein()` that generates a user specified length protein sequence in FASTA like format?

```
aa20 <- c("A","R","N","D","C","Q","E","G","H","I",
          "L","K","M","F","P","S","T","W","Y","V")
generate_protein <- function(n = 6) {
  paste(sample(aa20, size = n, replace = T), collapse = "")
}
```

```
generate_protein()
```

```
[1] "VDLILN"
```

Q. Use your new `generate_protein()` function to generate all sequences between length 6 and 12 amino acids in length and check if any of these are unique in nature (i.e found in the NR database at NCBI)?

```
generate_protein(6)
```

```
[1] "GEIIPM"
```

```
generate_protein(7)
```

```
[1] "AQMSHWR"
```

```
generate_protein(8)
```

```
[1] "MMWEAIFF"
```

```
generate_protein(9)
```

```
[1] "YRDWHSSD"
```

```
generate_protein(10)
```

```
[1] "RTGVMIVYND"
```

```
generate_protein(11)
```

```
[1] "STAYKYVWMNE"
```

```
generate_protein(12)
```

```
[1] "RNHFFIKLMESL"
```

Or we could do a `for()` loop:

```
for(i in 6:12) {  
  cat(">", i, "\n", sep = "")  
  cat ( generate_protein(i), "\n" )  
}
```

```
>6  
PLPCVN  
>7  
FWIAAQL  
>8  
HHASFTVK  
>9  
KHDNGLMAH  
>10  
REKAANERNM  
>11  
DCDWHANVNKK  
>12  
SLCKPKDRHEKA
```